BattleChain VulnerableVault Audit Report 2

Version 1.0

alva-p.xyz

March 28, 2026

- Prepared by: alva-p
- Lead Auditors: Álvaro Pineda

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
- High
- Informational

Protocol Summary

BattleChain is a Cyfrin testnet designed for practicing smart contract exploitation under Safe Harbor agreements. The protocol deploys a `VulnerableVault` contract that accepts ERC20 token deposits and allows users to withdraw their full balance at any time. A companion `MockToken` provides the ERC20 token with an additional transfer hook mechanism.

The audit covers the deployed instance on the BattleChain testnet (chain ID 627):

Contract	Address
MockToken (BCDT)	`0x1c0f607347f981C7328D190FD2DfB28a89385Cf0`
VulnerableVault	`0x44666bb5a9587f5d5b151f1e83C9Ab49f377E1D8`

Disclaimer

The alva-p team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

This audit was performed under the BattleChain Safe Harbor agreement (`0xCE2983b0102A0cE3DF059AAA19ddf0`). All recovered funds were distributed according to the agreed bounty terms: 90% returned to the protocol recovery address, 10% retained as bounty.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity.

Audit Details

Scope

```
./src/
 VulnerableVault.sol
 MockToken.sol
```

Roles

- **Owner** - Deployer of the protocol. Seeds the vault with initial liquidity.
- **User** - Any address that deposits tokens into the vault and can call `withdrawAll()`.
- **Attacker** - Any address that registers a transfer hook on MockToken and exploits the CEI violation.

Executive Summary

The audit identified a critical reentrancy vulnerability in `VulnerableVault` that, combined with two supporting weaknesses in `MockToken`, allows a complete drain of the vault token balance with zero capital requirement.

The attack was successfully executed on March 28, 2026, draining **100,000 BCDT** from the vault in 11 recursive iterations. Total gas used: 563,476 on BattleChain L2 (chain ID 627).

- **Exploit TX:** `0xe398344e52a4e7d0f2337d3cb473880750a1c03042fb6dec53f3d87b9c1eda27`
- **Recovery sent:** 99,000 BCDT
- **Bounty retained:** 11,000 BCDT

Issues found

Severity	Number of issues found
High	1
Medium	0
Low	0
Info/Gas	2
Total	3

Findings

High

[H-1] Reentrancy in `VulnerableVault::withdrawAll` via CEI violation and MockToken transfer hook allows complete vault drain with zero capital

Description

`VulnerableVault::withdrawAll` performs the token transfer (Interaction) **before** zeroing the caller's balance (Effect), violating the Checks-Effects-Interactions pattern:

```
function withdrawAll() external {
    uint256 amount = balances[msg.sender];
    require(amount > 0, "nothing to withdraw");

@>  TOKEN.transfer(msg.sender, amount); // Interaction FIRST

@>  balances[msg.sender] = 0;          // Effect SECOND -- too late
    emit Withdrawn(msg.sender, amount);
}
```

This CEI violation becomes exploitable because `MockToken::transfer` is overridden to call an arbitrary hook on the recipient contract after moving tokens:

```
function transfer(address to, uint256 amount) public override returns (bool) {
    bool success = super.transfer(to, amount);
    address hook = transferHooks[to];
    if (hook != address(0)) {
@>      ITransferHook(hook).onTokenTransfer(msg.sender, amount); // re-entry point
    }
    return success;
}
```

Additionally, `MockToken::mint` has no access control, allowing anyone to obtain seed tokens for free:

```
function mint(address to, uint256 amount) external {
    _mint(to, amount); // no access control
}
```

The combination of these three weaknesses enables a complete vault drain with no initial capital.

Impact

All tokens deposited in the vault can be stolen by any attacker. The vault holds protocol liquidity intended for legitimate users. Severity: **Critical** - likelihood high, impact high.

Proof of Concept

Attack flow executed on March 28, 2026:

1. Deploy `Attacker` contract pointing to vault and token.
2. Register `Attacker` as a transfer hook: `MockToken::setTransferHook(attacker)`.
3. Mint 10,000 BCDT for free: `MockToken::mint(attacker, 10_000e18)`.
4. Approve vault and deposit 10,000 BCDT, establishing `balances[attacker] = 10_000e18`.
5. Call `vault.withdrawAll()`. Vault sends 10,000 BCDT triggering `onTokenTransfer()`.

6. Inside `onTokenTransfer()`, re-enter `vault.withdrawAll()`. Because `balances[attacker]` is still `10_000e18`, vault sends another 10,000 BCDT.
7. Repeat recursively until `TOKEN.balanceOf(vault) == 0`.
8. After 11 iterations: 110,000 BCDT drained (100,000 vault + 10,000 seed).
9. Distribute: 90% (99,000 BCDT) to recovery, 10% (11,000 BCDT) as bounty.

```
vault.withdrawAll()
  vault sends 10,000 BCDT to attacker
    onTokenTransfer() -- balances[attacker] still 10,000e18
      vault.withdrawAll()
        vault sends 10,000 BCDT to attacker
          onTokenTransfer() -- balances[attacker] still 10,000e18
            ... x11 until vault = 0
balances[attacker] = 0   -- executed only once, after full drain
```

Recommended Mitigation

Apply the CEI pattern in `VulnerableVault::withdrawAll` by zeroing the balance **before** the transfer:

```
function withdrawAll() external {
    uint256 amount = balances[msg.sender];
    require(amount > 0, "nothing to withdraw");

+   balances[msg.sender] = 0;
+   emit Withdrawn(msg.sender, amount);

    TOKEN.transfer(msg.sender, amount);

-   balances[msg.sender] = 0;
-   emit Withdrawn(msg.sender, amount);
}
```

Additionally, consider adding a reentrancy guard as defense in depth:

```
+ import {ReentrancyGuard} from "@openzeppelin/contracts/utils/
    ReentrancyGuard.sol";

- contract VulnerableVault {
+ contract VulnerableVault is ReentrancyGuard {

-   function withdrawAll() external {
+   function withdrawAll() external nonReentrant {
```

For `MockToken`, restrict mint access and consider limiting who can register transfer hooks:

```
- function mint(address to, uint256 amount) external {
+ function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
```

```
}
```

Informational

[I-1] `MockToken::transferFrom` does not trigger the transfer hook, creating inconsistent behavior

`MockToken` overrides `transfer` to invoke hooks but does not override `transferFrom`. This means hooks are not called on `transferFrom` transfers, creating an inconsistency that could confuse integrators.

Recommended Mitigation: Override `transferFrom` with the same hook logic, or document the intentional asymmetry clearly in the NatSpec.

[I-2] `MockToken::mint` has no access control

Any address can mint an unlimited amount of tokens. While intentional for the tutorial context, in a production deployment this would allow complete token supply inflation with no cost.

Recommended Mitigation: Add `onlyOwner` or a role-based access control modifier to `mint`.